INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO

Estrategia de Articulación EnContexto · Departamento de Sistemas Digitales

Trabajo de consulta en parejas

SolarBI Pascual: Power BI y Grafana, gobernanza de datos, automatización ETL e IoT

Integrantes

Andres Felipe Asprilla Pedroza

Jorge Andrés Grisales Herrera

Curso: Inteligencia de Negocios · Grupo 01

Semestre 2026-II

Docente: Ramiro Grisales Montoya

Medellín, octubre de 2026

Tabla de contenido

1. Propósito **3**

2. Ficha del trabajo **3**

3. Parte A · Consulta **3**

Bloque 1 · Power BI y Grafana 3

Bloque 2 · Gobernanza de datos (data governance) 5

Bloque 3 · Automatización de procesos ETL 7

Bloque 4 · Internet de las Cosas (IoT) 9

Bloque 5 · Control de versiones 11

4. Parte B · Práctica guiada: los mismos datos en dos herramientas **12**

Paso 1 · Bronze: datos crudos 13

Paso 2 · Silver: reglas de calidad 13

Paso 3 · Gold: carga en PostgreSQL 14

Paso 4 · Power BI 15

Paso 5 · Grafana 16

Paso 6 · Reflexión 16

5. Parte C · Repositorio en GitHub **17**

6. Condiciones de entrega **18**

7. Rúbrica de evaluación **19**

8. Referencias **19**

1. Propósito

En SolarBI el cuadro de mando final se construye en Power BI. Sin embargo, en la industria el monitoreo de plantas y equipos suele hacerse con Grafana. En esta consulta compararán las dos herramientas sobre los mismos datos y darán sus primeros pasos en tres temas que sostienen el proyecto: gobernanza de datos, automatización de procesos ETL e Internet de las Cosas (IoT), trabajando con control de versiones en GitHub.

> **Enfoque del Grupo 01:** monitoreo operativo. Su práctica se centra en la potencia en el tiempo y en la detección de fallas.

2. Ficha del trabajo

Tabla 1. *Ficha del trabajo.*

Campo	Contenido
Curso / Grupo	Inteligencia de Negocios · Grupo 01
Modalidad	Parejas (dos integrantes). Una sola entrega por pareja.
Tiempo disponible	Dos (2) días a partir de la publicación en Google Classroom.
Fecha límite	_____
Valor	_____
Articulación	Proyecto de Aula SolarBI Pascual. Prepara el Entregable III (esquema estrella y flujo ETL, semana 12) y el Entregable V (cuadro de mando, semana 16).
Entrega	Un archivo PDF central + repositorio en GitHub con los archivos de soporte.

Indicadores de resultado de aprendizaje que moviliza: U2-a (lago de datos), U2-b (calidad de datos), U2-d (flujos ETL), U3-d (indicadores de desempeño), U3-e (cuadros de mando) y U2-e (responsabilidad frente a la información).

3. Parte A · Consulta

Respondan cada pregunta con sus propias palabras, en máximo 120 palabras, aplicándola al caso SolarBI e indicando la fuente. Las tablas, los diagramas y los fragmentos de código no cuentan en el límite.

Bloque 1 · Power BI y Grafana

> D1. ¿Qué es Grafana? Describan sus componentes: data sources, panels, dashboards, variables y alerting. Diferencien Grafana OSS de Grafana Cloud.

Grafana es una plataforma de visualización y monitoreo que permite consultar datos de diferentes fuentes y mostrarlos mediante paneles y dashboards. Los **data sources** son las fuentes donde están almacenados los datos; los **panels** muestran gráficos, tablas o indicadores; los **dashboards** agrupan varios panels; las **variables** permiten cambiar dinámicamente filtros como sitio o dispositivo; y **alerting**

permite generar alertas cuando se cumplen determinadas condiciones. Para SolarBI, Grafana puede utilizarse para monitorear la potencia de los inversores y detectar fallas. **Grafana OSS** es la versión de código abierto que se instala y administra localmente, mientras que **Grafana Cloud** es un servicio administrado en la nube.

D2. *Elaboren una tabla comparativa Power BI vs. Grafana con mínimo 8 criterios: propósito principal, usuario típico, conexión a los datos, modelo semántico y lenguaje (DAX frente a la consulta del origen), actualización y tiempo real, alertas, licenciamiento y costo, y control de acceso.*

Tabla 2. *Comparación entre Power BI y Grafana.*

Criterio	Power BI	Grafana
Propósito	Análisis y toma de decisiones	Monitoreo y observabilidad
Usuario típico	Analistas y directivos	Operadores y equipos técnicos
Datos	Múltiples fuentes	Principalmente consultas a fuentes de datos
Modelo/lenguaje	Modelo semántico y DAX	Consulta del origen de datos
Actualización	Programada y otras opciones según servicio	Orientado a datos actuales y series de tiempo
Alertas	Alertas sobre determinados datos del servicio	Alert rules y contact points
Costo	Licencias según modalidad	OSS gratuito; Cloud según servicio
Acceso	Roles, permisos y RLS	Organizaciones, equipos y permisos
SolarBI	KPIs y análisis empresarial	Monitoreo operativo de la planta

D3. *Diferencien un dashboard operativo, uno analítico y uno estratégico. Clasifiquen cada KPI mínimo del proyecto (energía, yield y Performance Ratio, ahorro, disponibilidad y alarmas, porcentaje de datos válidos) e indiquen en cuál de las dos herramientas lo publicarían y por qué.*

Un dashboard **operativo** permite supervisar lo que ocurre actualmente y detectar problemas rápidamente. Uno **analítico** permite estudiar tendencias y relaciones para comprender el comportamiento de los datos. Uno **estratégico** resume indicadores de alto nivel para apoyar decisiones directivas. En SolarBI, la **potencia, disponibilidad y alarmas** son principalmente operativas y pueden publicarse en Grafana. La **energía, yield y Performance Ratio** pueden analizarse en Power BI para estudiar el desempeño de la planta. El **ahorro** también puede presentarse en Power BI por su relación con resultados y decisiones del negocio. El **porcentaje de datos válidos** sirve para evaluar la calidad de la información y puede mostrarse en Power BI y utilizarse como indicador de confiabilidad.

> **D4.** *Grafana con PostgreSQL: ¿cómo debe ser una consulta de series de tiempo y para qué sirven las macros $__timeFilter y $__timeGroup? Escriban la consulta de la potencia promedio por hora.*

Una consulta de series de tiempo en Grafana debe devolver una columna temporal que Grafana pueda reconocer como eje de tiempo y una o más columnas con los valores que se desean representar. La macro **$__timeFilter** permite limitar los registros al intervalo seleccionado en el dashboard. **$__timeGroup** permite agrupar las fechas en intervalos, por ejemplo horas, minutos o días. Para obtener la potencia promedio por hora en PostgreSQL se puede utilizar:

```
SELECT
  $__timeGroup(ts, '1h') AS time,
  AVG(p_ac_kw) AS potencia_promedio_kw
FROM silver.lectura_5min
WHERE $__timeFilter(ts)
GROUP BY 1
ORDER BY 1;
```

> **D5.** *Alertas: comparen Grafana Alerting (alert rules y contact points) con las alertas de datos del servicio de Power BI. Propongan una regla de alerta útil para la planta y el canal por el que se notificaría.*

Grafana Alerting permite crear **alert rules** que evalúan condiciones sobre los datos y utilizar **contact points** para definir dónde enviar las notificaciones, por ejemplo correo electrónico. En Power BI también existen alertas de datos, principalmente asociadas a determinados elementos visuales y valores que cumplen una condición configurada en el servicio. Para SolarBI proponemos una alerta en Grafana que detecte que la potencia del inversor permanece en **cero entre las 9:00 y las 15:00**, cuando normalmente debería existir generación solar. Esta condición podría indicar una falla del inversor, una desconexión o un problema de datos. La notificación podría enviarse por correo al responsable de operación para que pueda revisar el equipo. Esta regla coincide con la práctica solicitada posteriormente en el trabajo.

Bloque 2 · Gobernanza de datos (data governance)

> **G1.** *Definan data governance y diferéncienla de data management y de data quality. Tomen como referencia el marco DAMA-DMBOK.*

La **gobernanza de datos (data governance)** es el conjunto de políticas, reglas, responsabilidades y procesos que permiten administrar los datos de manera organizada, segura y confiable. Según el enfoque de DAMA-DMBOK, la gobernanza establece quién puede tomar decisiones sobre los datos y bajo qué normas deben gestionarse. En SolarBI, significa definir quién es responsable de las mediciones de los dispositivos, cómo deben almacenarse, qué estándares deben cumplir y quién puede utilizarlas. Se diferencia del **data management**, que comprende las actividades y herramientas utilizadas para gestionar los datos, y de **data quality**, que se enfoca en evaluar y mejorar características como exactitud, completitud, consistencia y validez. La gobernanza establece las reglas; la gestión las ejecuta y la calidad verifica sus resultados.

Fuente: DAMA International, *DAMA-DMBOK2: Data Management Body of Knowledge*.

G2. *Expliquen los roles data owner, data steward y data custodian. Relaciónenlos con los roles del equipo en el proyecto (data engineer, data modeler, BI analyst y product owner).*

El **data owner** es quien tiene la responsabilidad sobre un conjunto de datos y toma decisiones relacionadas con su uso, acceso y reglas. El **data steward** se encarga de aplicar y supervisar las políticas de calidad, definición y uso correcto de los datos. El **data custodian** administra técnicamente los sistemas donde se almacenan y protegen los datos. En SolarBI, el **product owner** podría asumir responsabilidades de data owner al definir necesidades y reglas del producto; el **BI analyst** puede desempeñar funciones de data steward al revisar indicadores y calidad; y el **data engineer** actuaría como data custodian al implementar procesos de almacenamiento, seguridad y transformación. El **data modeler** apoyaría la definición de estructuras y relaciones de los datos.

Fuente: DAMA International, *DAMA-DMBOK2*.

G3. *El contrato de datos (data contract) del proyecto como instrumento de gobernanza: ¿qué debe incluir (esquema, unidades, frecuencia, responsable, niveles de calidad)? ¿Qué le pasa al cuadro de mando si la fuente cambia un campo sin avisar?*

El **data contract** establece las condiciones que debe cumplir una fuente para que los datos puedan utilizarse correctamente. En SolarBI debería especificar el **esquema de los datos**, nombres y tipos de campos, unidades de medida, frecuencia de envío, responsable de la información y niveles mínimos de calidad, como porcentaje de datos válidos, valores permitidos y manejo de faltantes. También ayuda a mantener un acuerdo entre quienes producen y consumen los datos. Si una fuente cambia un campo sin avisar, por ejemplo modifica el nombre o tipo de una columna, el proceso ETL puede fallar o producir información incorrecta. Como consecuencia, las tablas y los indicadores del cuadro de mando podrían quedar incompletos, presentar errores o dejar de actualizarse.

Fuente: DAMA International, *DAMA-DMBOK2*; documentación del trabajo SolarBI. La guía plantea específicamente estos elementos para el contrato de datos.

G4. *Seguridad a nivel de fila (row-level security, RLS) en Power BI: roles y filtros DAX. Den el ejemplo de un responsable que solo ve su sitio. ¿Cuál es el mecanismo equivalente de control de acceso en Grafana (organizaciones, equipos y permisos por carpeta o dashboard)?*

La **seguridad a nivel de fila (RLS)** permite limitar los registros que puede consultar cada usuario dentro de un modelo de datos. En Power BI se crean roles y se aplican filtros mediante DAX. Por ejemplo, si SolarBI tiene información de varios sitios, un responsable podría tener un rol que filtre los datos para mostrar únicamente los registros correspondientes a su sitio. Así, aunque todos los datos estén almacenados en el mismo modelo, cada usuario visualiza únicamente la información autorizada. En Grafana, el control de acceso se organiza mediante **organizaciones, equipos y permisos** asignados a carpetas y dashboards. De esta forma se puede restringir quién puede visualizar o modificar determinados tableros. Ambos mecanismos buscan evitar accesos no autorizados, aunque funcionan de manera diferente.

Fuente: Microsoft Learn, *Row-level security (RLS) with Power BI*; Grafana Documentation. La guía solicita comparar ambos mecanismos.

G5. *Ley 1581 de 2012 (protección de datos personales en Colombia): ¿los datos de SolarBI son personales? ¿Qué cambiaría si el tablero mostrara qué operador atendió cada alarma? Fijen la postura del equipo frente al manejo responsable de la información.*

Los datos de generación eléctrica de SolarBI, como potencia, irradiancia o energía producida, **no son necesariamente datos personales** si únicamente representan información técnica de los equipos y no permiten identificar directamente a una persona. Sin embargo, la situación cambia si el tablero registra qué operador atendió cada alarma, porque el nombre o identificación del operador puede constituir un dato personal. En ese caso se deben aplicar medidas de protección, acceso autorizado y tratamiento responsable de la información conforme a la Ley 1581 de 2012. La postura del equipo debe ser utilizar únicamente los datos necesarios para el propósito del proyecto, limitar los accesos, proteger la información personal y evitar exponer datos de trabajadores sin una finalidad justificada.

Fuente: Ley 1581 de 2012, República de Colombia. La pregunta del trabajo plantea expresamente esta situación.

Bloque 3 · Automatización de procesos ETL

E1. *Diferencien ETL de ELT y ubiquen cada paso en las capas Bronze, Silver y Gold del proyecto. ¿Power Query es una herramienta ETL? Argumenten.*

ETL significa extraer, transformar y cargar: los datos se transforman antes de almacenarse en el destino. En **ELT**, primero se extraen y cargan los datos y después se transforman dentro del sistema de almacenamiento. En SolarBI, las capas pueden organizarse como **Bronze**, donde se conserva el dato crudo sin modificar; **Silver**, donde se aplican reglas de limpieza y calidad; y **Gold**, donde se generan datos preparados para análisis, como los resúmenes diarios de energía. **Power Query sí es una herramienta ETL**, porque permite extraer datos de diferentes fuentes, transformarlos mediante operaciones de limpieza y finalmente cargarlos en un modelo o destino. Su uso permite preparar los datos antes de utilizarlos en Power BI.

E2. *Comparen carga completa (full load) y carga incremental. ¿Qué significa que una carga sea idempotente y cómo se logra en fact_energia_dia con su clave primaria (fecha_key, dispositivo_key) y un UPSERT?*

Una **carga completa (full load)** vuelve a procesar y cargar todo el conjunto de datos, mientras que una **carga incremental** procesa únicamente los registros nuevos o modificados desde la última ejecución. La carga incremental puede ser más eficiente cuando el volumen de información aumenta. Una carga es **idempotente** cuando ejecutarla varias veces produce el mismo resultado que ejecutarla una sola vez, evitando registros duplicados. En fact_energia_dia, la clave primaria **(fecha_key, dispositivo_key)** identifica de forma única el resumen de cada dispositivo para cada fecha. Mediante un **UPSERT**, si la combinación no existe se inserta el registro; si ya existe, se actualiza. Así, ejecutar nuevamente el ETL no genera duplicados y mantiene los datos consistentes.

E3. *Elaboren una tabla comparativa de tres formas de automatizar un flujo: cron (o el Programador de tareas de Windows), Apache Airflow y la actualización programada de Power BI. Incluyan qué es, qué automatiza cada una y cuándo conviene.*

Tabla 3. *Comparación de tres formas de automatizar un flujo ETL.*

Criterio	Cron / Programador de tareas	Apache Airflow	Actualización programada de Power BI
¿Qué es?	Programador de tareas del sistema operativo	Plataforma de orquestación de flujos	Función del servicio Power BI
¿Qué automatiza?	Ejecución de comandos o scripts	Flujos completos con varias tareas y dependencias	Actualización de datasets/modelos
Complejidad	Baja	Alta	Baja
Control de dependencias	Limitado	Amplio	Limitado al proceso de actualización
Registro y seguimiento	Básico	Avanzado	Disponible en el servicio
Cuándo conviene	ETL pequeños, como el de SolarBI	Procesos complejos con muchas tareas	Cuando los datos ya están preparados y solo se necesita actualizar Power BI

Conclusión: Para el flujo inicial de SolarBI, **cron o el Programador de tareas de Windows** resulta suficiente para ejecutar etl/run_etl.py diariamente. Airflow sería más conveniente si el proyecto creciera y tuviera múltiples procesos dependientes. La actualización programada de Power BI es apropiada cuando el ETL ya está resuelto y únicamente se necesita actualizar el modelo o conjunto de datos.

E4. *Query folding en Power Query: ¿qué es, por qué mejora el rendimiento y cómo se verifica si un paso se está plegando hacia PostgreSQL?*

El **query folding** es el proceso mediante el cual Power Query intenta convertir las transformaciones realizadas en una consulta en instrucciones que puedan ejecutarse directamente en el origen de datos, en este caso PostgreSQL. Esto mejora el rendimiento porque parte del procesamiento se realiza en el servidor y no necesariamente en el equipo del usuario, reduciendo los datos que deben transferirse y procesarse localmente. Para SolarBI, esto puede ser útil al trabajar con grandes cantidades de lecturas de telemetría. Para verificar si un paso se está plegando, en Power Query se puede utilizar la opción **"Ver consulta nativa"** cuando esté disponible. También se puede revisar el comportamiento de los pasos posteriores para identificar dónde se pierde el plegado.

Bloque 4 · Internet de las Cosas (IoT)

I1. *Describan la arquitectura IoT por capas: dispositivo, gateway (edge), red o broker, almacenamiento y aplicación. Dibujen cómo viaja una lectura del inversor hasta llegar a un visual del tablero, indicando en qué capa (Bronze, Silver, Gold) queda en cada momento.*

La arquitectura IoT reparte el trabajo en cinco capas. El **dispositivo** (el inversor de 5 kWp y sus sensores) mide la potencia, la irradiancia y la temperatura del módulo. El **gateway o edge** (un ESP32 o una Raspberry Pi que lee el inversor por Modbus) recoge las lecturas, les asigna una marca de tiempo y hace un primer filtro. La **red o broker** (MQTT sobre Wi-Fi) transporta los datos y separa a quien los produce de quien los consume. El **almacenamiento** guarda la lectura cruda (Bronze), la limpia (Silver) y la agrega (Gold). La **aplicación** (Power BI y Grafana) la convierte en visuales. En nuestra práctica, simulador.py reemplaza las tres primeras capas (AWS, s.f.; Microsoft, s.f.).

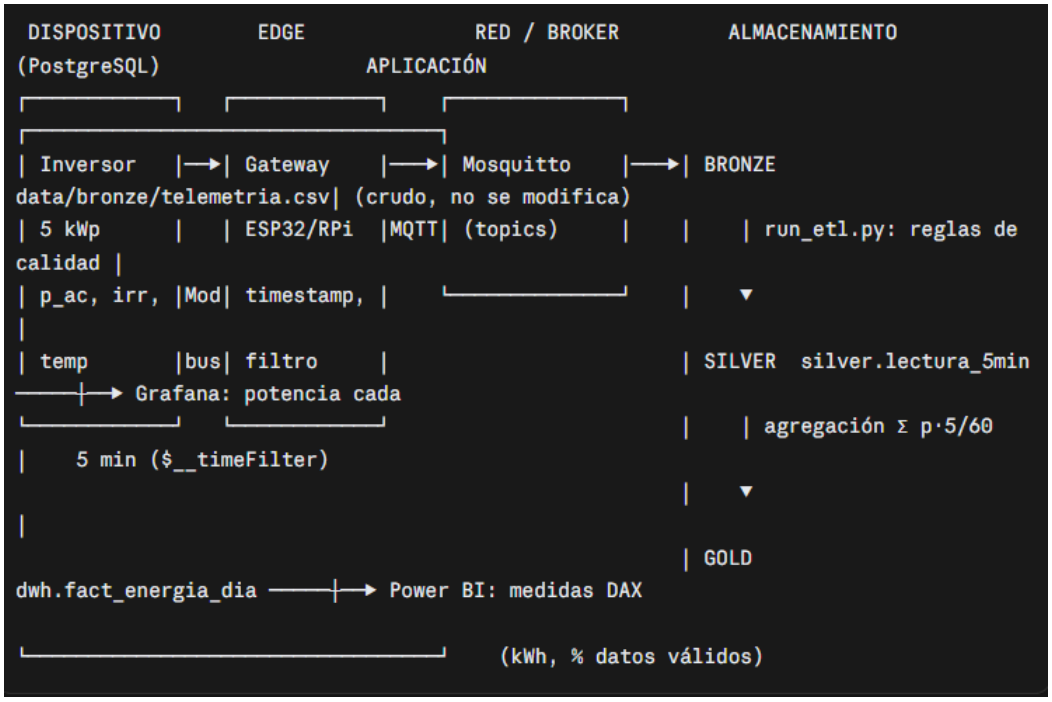


Figura 1. *Recorrido de una lectura del inversor por las capas de la arquitectura IoT y por las capas Bronze, Silver y Gold.*

I2. *Expliquen MQTT: modelo publish/subscribe, broker, topic y niveles de QoS (0, 1 y 2). Propongan la jerarquía de topics de la planta, por ejemplo pascualbravo/solar/<sitio>/<dispositivo>/<variable>.*

MQTT es un protocolo de mensajería liviano, estandarizado por OASIS y pensado para dispositivos y redes con pocos recursos. Funciona con **publish/subscribe**: el gateway publica en un **topic** sin saber quién va a leer, y el **broker** (Mosquitto) entrega el mensaje a los suscriptores (Telegraf o el ETL). El topic es una ruta jerárquica separada por / que acepta comodines: + para un nivel y # para varios. Los niveles de **QoS** son tres: **0** entrega como máximo una vez y no confirma; **1** entrega al menos una vez y puede duplicar; **2** entrega exactamente una vez, con un intercambio de cuatro pasos que cuesta más. Para

SolarBI proponemos **QoS 1** en la telemetría, porque los duplicados ya se eliminan en Silver, y **QoS 2** en las alarmas (OASIS, 2019; HiveMQ, s.f.).

Topic	Contenido	QoS
`pascualbravo/solar/robledo/inv01/p_ac_kw`	Potencia AC cada 5 min	1
`pascualbravo/solar/robledo/inv01/irradiancia_wm2`	Irradiancia	1
`pascualbravo/solar/robledo/inv01/temp_modulo_c`	Temperatura del módulo	1
`pascualbravo/solar/robledo/inv01/estado`	online/offline (retained + Last Will)	1
`pascualbravo/solar/robledo/inv01/alarma`	Código de falla	2

Figura 2. *Jerarquía de topics MQTT propuesta para la planta y nivel de QoS de cada variable.*

Ejemplos de suscripción: pascualbravo/solar/+/+/p_ac_kw trae la potencia de todos los sitios y equipos, y pascualbravo/solar/robledo/# trae todo lo de la sede.

I3. *Describan una pila abierta típica de monitoreo IoT: Mosquitto, Telegraf, una base de series de tiempo (InfluxDB o TimescaleDB) y Grafana. ¿Qué papel cumple cada componente y dónde encajaría PostgreSQL?*

La pila separa la ingesta, el almacenamiento y la visualización. **Mosquitto** es el broker MQTT que recibe la telemetría del gateway. **Telegraf** es un agente que se suscribe a los topics con el plugin mqtt_consumer, interpreta los mensajes y los escribe por lotes en la base. Una **base de series de tiempo** (InfluxDB o TimescaleDB) está optimizada para escribir muchos datos indexados por tiempo, manejar retención y agregar por ventanas. **Grafana** consulta esa base y construye paneles y alertas. **PostgreSQL** encaja directamente porque TimescaleDB es una extensión suya: silver.lectura_5min puede ser una *hypertable* y dwh.fact_energia_dia puede vivir en la misma instancia. Así, Grafana y Power BI leen una sola fuente de verdad (Eclipse Foundation, s.f.; InfluxData, s.f.; Timescale, s.f.).

Componente	Papel	En SolarBI
Mosquitto	Broker MQTT	Recibe los topics `pascualbravo/solar/...`
Telegraf	Agente de ingesta	De MQTT a la base (`outputs.postgresql`)
InfluxDB / TimescaleDB	Base de series de tiempo	TimescaleDB sobre PostgreSQL (Silver)
Grafana	Visualización y alertas	Panel de potencia y alerta de potencia cero entre 9:00 y 15:00
PostgreSQL	Base relacional	Silver + Gold; Power BI se conecta aquí

Figura 3. *Pila abierta de monitoreo IoT y papel de cada componente en SolarBI.*

> **I4.** *Procesamiento por lotes (batch) frente a flujo continuo (streaming). La telemetría llega cada 5 minutos y fact_energia_dia tiene granularidad diaria: ¿qué se gana y qué se pierde al agregar? ¿Qué herramienta atiende mejor cada granularidad?*

El procesamiento **batch** trabaja sobre datos acumulados en un intervalo, por ejemplo run_etl.py a medianoche. El **streaming** procesa cada evento apenas llega, con latencia de segundos. Al agregar las 288 lecturas de 5 minutos en una sola fila diaria de fact_energia_dia (energía = Σ p_ac_kw × 5/60) **se gana** menos volumen, consultas más rápidas, KPIs estables para comparar días (yield, PR, ahorro) y un modelo más simple en Power BI. **Se pierde** la forma de la curva, los picos, las caídas breves y el momento exacto de una falla: una potencia en cero durante 20 minutos a las 11:00 desaparece dentro del total del día. Por eso **Grafana** atiende la granularidad de 5 minutos (uso operativo y detección de fallas) y **Power BI** la diaria (uso analítico y estratégico) (Kleppmann, 2017).

Tabla 4. *Granularidad de 5 minutos frente a granularidad diaria.*

	5 minutos (Silver)	Diaria (Gold)
Filas en 3 días	864	3
Detecta fallas puntuales	Sí	No
Tendencias y KPIs	Ruidoso	Claro
Herramienta	Grafana	Power BI

Bloque 5 · Control de versiones

> **V1.** *Diferencien Git de GitHub y definan repository, commit, branch y pull request. Un archivo .pbix es binario: ¿qué problema genera en Git y qué resuelve el formato Power BI Project (.pbip)? ¿Cómo se versiona un dashboard de Grafana?*

Git es el sistema de control de versiones que corre en nuestro equipo y registra cada cambio. **GitHub** es la plataforma en la nube que aloja el repositorio y agrega la colaboración: colaboradores, pull requests e issues. Un **.pbix** es un archivo binario comprimido, así que Git no puede mostrar qué cambió ni combinar las ediciones de los dos integrantes; solo guarda copias completas y pesadas. El formato **.pbip** divide el proyecto en carpetas de texto (el modelo semántico en TMDL y el reporte en JSON/PBIR). Con eso cada medida DAX aparece como un diff legible, y la caché de datos se excluye con .gitignore. En **Grafana** el dashboard ya es JSON: se exporta a grafana/dashboard.json, se versiona como código y se puede volver a cargar con *provisioning* (Chacon y Straub, 2014; Microsoft, s.f.; Grafana Labs, s.f.).

Concepto	Definición	Ejemplo en SolarBI
Repository	Carpeta del proyecto con todo su historial de cambios	`solarbi-apellido1-apellido2`
Commit	Foto del proyecto en un momento, con autor y mensaje	`feat(etl): agrega regla de rango físico para p_ac_kw`
Branch	Línea de trabajo paralela que no afecta a `main`	`feature/grafana-dashboard`
Pull request	Solicitud para revisar y fusionar una rama en `main`	El compañero revisa la consulta SQL antes de fusionar

Aspecto	.pbix	.pbip
Tipo de archivo	Binario (zip)	Carpeta de archivos de texto
Diff en Git	No legible	Línea por línea (TMDL/JSON)
Merge entre integrantes	Imposible, gana la última copia	Posible
Tamaño en el repositorio	Incluye los datos y crece en cada commit	Solo la definición; los datos se excluyen

Figura 4. *Conceptos de Git y GitHub aplicados a SolarBI, y comparación entre los formatos .pbix y .pbip.*

Flujo de trabajo:

```
git checkout -b feature/grafana-dashboard
git add grafana/dashboard.json
git commit -m "feat(grafana): panel de potencia 5 min con alerta de potencia cero 9-15h"
git push origin feature/grafana-dashboard   # luego se abre el pull request en GitHub
```

.gitignore mínimo:

```
.env
**/.pbi/localSettings.json
**/.pbi/cache.abf
__pycache__/
```

4. Parte B · Práctica guiada: los mismos datos en dos herramientas

Construirán un flujo pequeño pero completo: datos crudos de un dispositivo simulado, limpieza con reglas de calidad, carga en PostgreSQL y dos visualizaciones sobre esa misma base, una en Power BI y otra en Grafana.

Paso 1 · Bronze: datos crudos

Usen el dataset del contrato de datos del curso o, si lo prefieren, este simulador base (etl/simulador.py), que genera 3 días de telemetría de un inversor de 5 kWp con anomalías inyectadas. El archivo de Bronze no se modifica.

```
import csv, math, os, random
from datetime import datetime, timedelta
inicio = datetime(2026, 10, 5, 0, 0)
os.makedirs("data/bronze", exist_ok=True)
with open("data/bronze/telemetria.csv", "w", newline="", encoding="utf-8") as f:
    w = csv.writer(f)
    w.writerow(["ts", "dispositivo_id", "p_ac_kw", "irradiancia_wm2", "temp_modulo_c"])
    for i in range(3 * 288):                          # 3 dias, cada 5 min
        ts = inicio + timedelta(minutes=5 * i)
        sol = max(0.0, math.sin(math.pi * (ts.hour + ts.minute / 60 - 6) / 12))
        irr = round(1000 * sol * random.uniform(0.7, 1.0), 1)
        p_ac = round(5.0 * irr / 1000 * random.uniform(0.80, 0.90), 3)
        fila = [ts.isoformat(sep=" "), 1, p_ac, irr, round(22 + 30 * sol, 1)]
        r = random.random()
        if r < 0.02:
            fila[2] = -1                              # anomalia: potencia negativa
        elif r < 0.04:
            fila[3] = ""                              # anomalia: dato faltante
        w.writerow(fila)
        if r > 0.98:
            w.writerow(fila)                          # anomalia: fila duplicada
```

Paso 2 · Silver: reglas de calidad

Con Python (pandas) o con SQL, apliquen mínimo tres reglas de calidad: rango físico válido, duplicados y datos faltantes. Reporten filas leídas, filas rechazadas por cada regla, filas válidas y el porcentaje de datos válidos.

Resultados

Concepto	Filas
Filas leídas	881
Rechazadas por datos faltantes	14
Rechazadas por rango físico inválido	16
Rechazadas por duplicados	17
Filas válidas	**834**
Porcentaje de datos válidos	**94.67 %**

Figura 5. *Resultados de las reglas de calidad aplicadas en la capa Silver.*

Se aplicó el proceso de limpieza con Python (pandas) sobre el archivo `data/bronze/telemetria.csv`, que contiene la telemetría simulada de un inversor solar con lecturas cada 5 minutos durante 3 días. Se implementaron tres reglas de calidad, aplicadas en orden:

1. **Datos faltantes:** se eliminan las filas con valores nulos en alguna columna clave (`ts`, `dispositivo_id`, `p_ac_kw`, `irradiancia_wm2`, `temp_modulo_c`). Se rechazaron 14 filas.
2. **Rango físico válido:** la potencia (`p_ac_kw`) y la irradiancia (`irradiancia_wm2`) no pueden ser negativas. Se rechazaron 16 filas, por ejemplo lecturas con potencia de -1 kW.
3. **Duplicados:** se descartan las filas exactamente repetidas. Se rechazaron 17 filas.

De las 881 filas leídas quedaron 834 filas válidas, es decir, el **94,67 %** de los datos. Estas filas pasan a la capa Silver y se usan para calcular la energía diaria (kWh) de la capa Gold. Las anomalías se introdujeron a propósito en el simulador para comprobar que las reglas las detectan.

Paso 3 · Gold: carga en PostgreSQL

- Carguen las lecturas limpias de 5 minutos en una tabla silver.lectura_5min.
- Carguen el resumen diario en una tabla con la estructura de dwh.fact_energia_dia (como mínimo: fecha, dispositivo, energia_kwh y pct_datos_validos). Recuerden que la energía de cada intervalo es la potencia por 5/60 de hora.
- Dejen todo en un solo punto de entrada (etl/run_etl.py) que se ejecute con un único comando. Ejecútenlo dos veces y demuestren que los conteos no cambian (carga idempotente).
- Escriban, sin necesidad de implementarla, la expresión cron (o la configuración del Programador de tareas) que lanzaría ese comando cada día a la medianoche.
- **Carga Silver.** Las 834 lecturas limpias de 5 minutos se cargan en la tabla `silver.lectura_5min`, con clave primaria `(ts, dispositivo_id)`.
- **Carga Gold.** El resumen diario se guarda en `dwh.fact_energia_dia` con las columnas `fecha_key`, `dispositivo_key`, `energia_kwh` y `pct_datos_validos`. La energía de cada intervalo se calcula como potencia × 5/60 horas, y se suma por día y por dispositivo.
- **Un solo punto de entrada.** Todo el proceso se ejecuta con un único comando: `python etl/run_etl.py`.
- **Idempotencia.** Ambas tablas tienen clave primaria y usan `INSERT ... ON CONFLICT DO UPDATE`. Si una fila ya existe, se actualiza en lugar de duplicarse. Al ejecutar el ETL dos veces, los conteos se mantuvieron en 834 filas en Silver y 3 en Gold.

Prueba de idempotencia

Tabla	Conteo 1.ª ejecución	Conteo 2.ª ejecución	Resultado
silver.lectura_5min	834	834	Igual ✔
dwh.fact_energia_dia	3	3	Igual ✔

dwh.fact_energia_dia

fecha_key	dispositivo_key	energia_kwh	pct_datos_validos
2026-10-05	1	26.439	94.67
2026-10-06	1	26.551	94.67
2026-10-07	1	26.801	94.67

Figura 6. *Prueba de idempotencia: conteos de silver.lectura_5min y dwh.fact_energia_dia tras dos ejecuciones del ETL.*

Paso 4 · Power BI

Conecten Power BI Desktop a PostgreSQL y construyan una página con: una tarjeta con la energía total (kWh), un gráfico de líneas con la energía diaria y una tarjeta con el porcentaje de datos válidos. Las cifras deben salir de medidas DAX, no de columnas arrastradas.

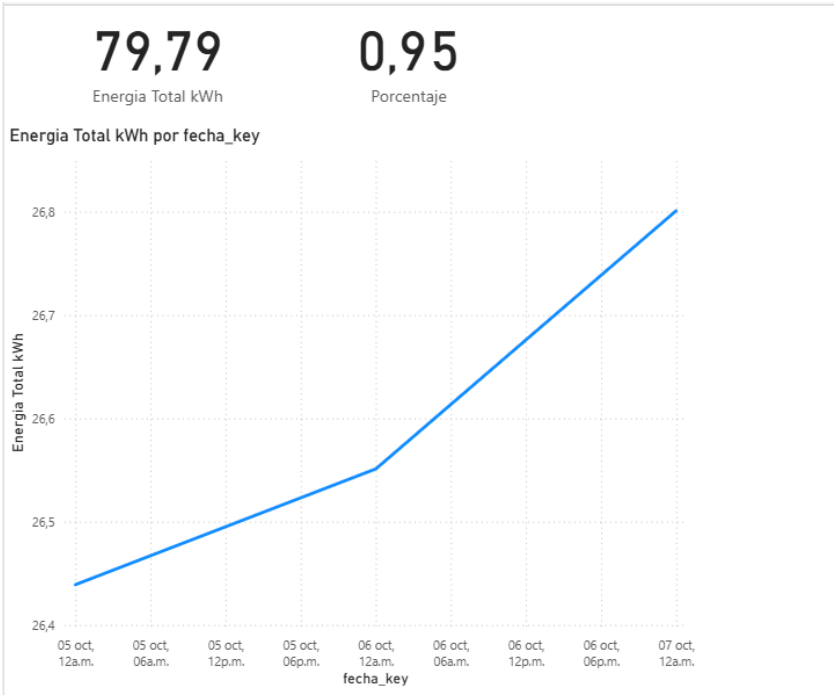


Figura 7. *Página de Power BI con la energía total, el porcentaje de datos válidos y la energía diaria.*

Se construyó una página en Power BI Desktop conectada a la tabla dwh.fact_energia_dia. Incluye una tarjeta con la energía total (kWh), un gráfico de líneas con la energía diaria y una tarjeta con el porcentaje de datos válidos. Las tres cifras provienen de medidas DAX (Energia Total kWh y Pct Datos Validos), no de columnas arrastradas directamente al lienzo.

Paso 5 · Grafana

Instalen Grafana OSS en su equipo (instalador o Docker), agreguen PostgreSQL como data source y construyan un dashboard con:

- Un panel de series de tiempo con la potencia p_ac_kw cada 5 minutos, usando $__timeFilter en la consulta.
- Un umbral (threshold) o una regla de alerta que se active cuando la potencia sea cero entre las 9:00 y las 15:00.
- Un panel tipo stat con la energía del día seleccionado.
- Exporten el dashboard como JSON y guárdenlo en el repositorio.

Si no logran instalar Grafana, documenten el error con capturas, expliquen qué intentaron y exploren los tableros de ejemplo de play.grafana.org para sustentar su comparación. Esta ruta alterna se evalúa con menor puntaje en el criterio de tableros.

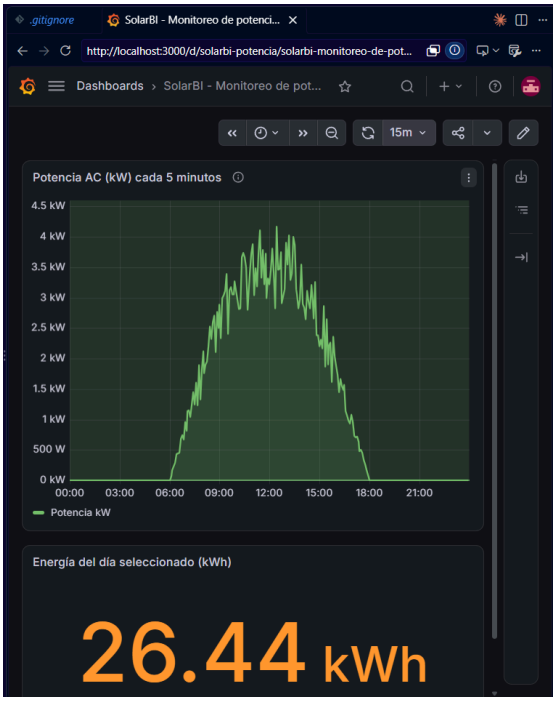


Figura 8. *Dashboard de Grafana con la potencia AC cada 5 minutos y la energía del día seleccionado.*

Paso 6 · Reflexión

En máximo 10 líneas: con los mismos datos, ¿qué resolvió mejor cada herramienta?, ¿qué les costó más?, ¿cuál usarían para el operador de la planta y cuál para un directivo?

Qué resolvió mejor cada una

- **Power BI:** resumen ejecutivo claro, con tarjetas de energía total (79,79 kWh) y % de datos válidos (94,67 %), y medidas DAX reutilizables.
- **Grafana:** monitoreo cada 5 minutos con `$__timeFilter` y la alerta de potencia cero entre las 9:00 y las 15:00.

Qué costó más

- **Power BI:** configuración regional (26.439 leído como 26439), convertir la fecha en jerarquía y dar formato a las medidas.
- **Grafana:** instalar PostgreSQL y Grafana, conectar la fuente de datos y escribir el SQL y la alerta a mano.

Quién usa cuál

- **Operador de la planta:** Grafana, porque ve la curva casi en tiempo real y recibe alertas cuando algo falla.
- **Directivo:** Power BI, porque resume en pocas cifras y es fácil de compartir y explicar.

> **Evidencias de la Parte B en el PDF:** una captura por paso con una explicación de dos o tres líneas, incluidas las capturas de los dos tableros. El código va en el repositorio, no pegado completo en el PDF.

5. Parte C · Repositorio en GitHub

Todo el trabajo se desarrolla y se entrega integrado a un repositorio en GitHub. Este repositorio no es de un solo uso: será la base sobre la que continuarán los siguientes trabajos del proyecto hasta el final del semestre.

- Un integrante crea el repositorio **público** con el nombre solarbi-apellido1-apellido2.
- Agrega a su compañero como **colaborador** (Settings → Collaborators → Add people). El compañero debe aceptar la invitación antes de la entrega.
- Los dos integrantes deben aportar: mínimo **3 commits por persona**, con mensajes descriptivos (no "update" ni "cambios").
- El README.md incluye: nombres completos, curso y grupo, descripción del trabajo, pasos para reproducir la práctica y una tabla breve de quién hizo qué.
- Incluyan un .gitignore. **Nunca** suban contraseñas ni cadenas de conexión; usen variables de entorno o un archivo .env excluido del repositorio.

Estructura mínima del repositorio:

```
solarbi-apellido1-apellido2/
├── README.md
├── .gitignore
├── docs/       PDF de la consulta
├── data/       bronze/ (crudo) y silver/ (limpio)
├── etl/        simulador.py, run_etl.py
├── sql/        tablas y cargas en PostgreSQL
├── powerbi/    archivo .pbix o proyecto .pbip
└── grafana/    dashboard.json exportado
```

Evidencias que deben aparecer en el PDF: enlace al repositorio, captura de la sección Collaborators con los dos integrantes y captura del historial de commits donde se vean ambos autores.

Enlace al repositorio: https://github.com/JorgeAnDev/SolarBI-JorgeGrisales-AndresAsprilla.git

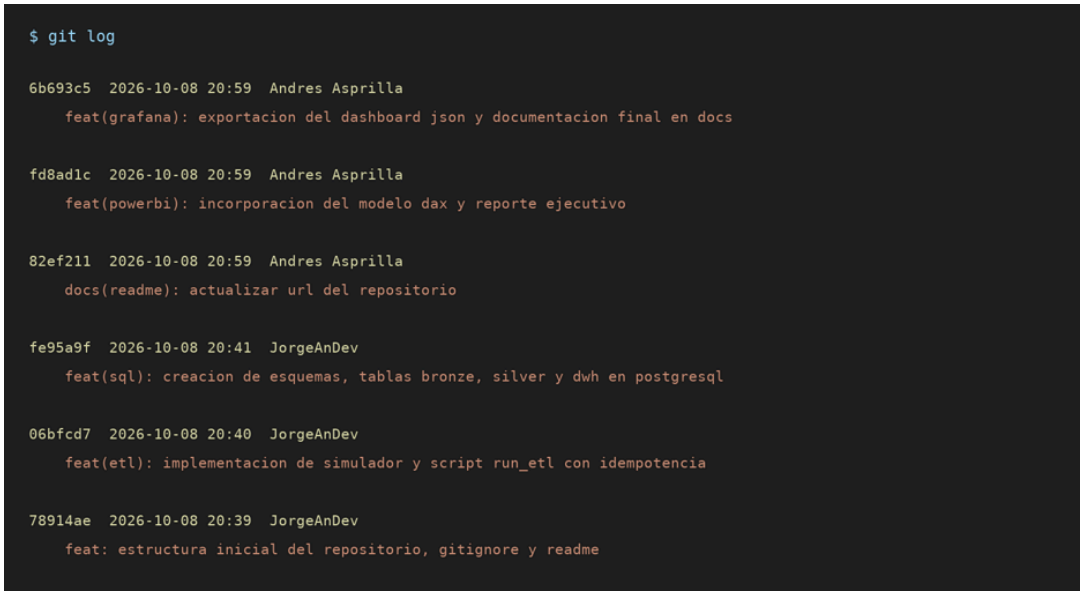


Figura 9. *Historial de commits del repositorio (git log), en el que se observan ambos autores.*

7. Rúbrica de evaluación

Tabla 5. *Rúbrica de evaluación.*

Criterio	Descriptor de nivel alto	Peso
Consulta (Parte A)	Respuestas correctas, en palabras propias, aplicadas a SolarBI y con fuentes confiables citadas.	35%
Flujo ETL (Parte B)	Reglas de calidad documentadas, carga idempotente en PostgreSQL con un solo comando y métricas de calidad reportadas.	20%
Tableros (Parte B)	Power BI con medidas DAX y Grafana con consulta de series de tiempo, ambos sobre la misma base, y una reflexión comparativa bien argumentada.	20%
GitHub (Parte C)	Repositorio ordenado, README reproducible, compañero como colaborador y commits de ambos integrantes.	15%
Presentación y ética	PDF con el nombre exigido, bien organizado, con referencias y sin credenciales expuestas.	10%

8. Referencias

Amazon Web Services. (s.f.). *Documentación de AWS IoT*. https://docs.aws.amazon.com/iot/

Chacon, S. y Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book

Congreso de la República de Colombia. (2012). *Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*.

DAMA International. (2017). *DAMA-DMBOK: Data Management Body of Knowledge* (2.ª ed.). Technics Publications.

Eclipse Foundation. (s.f.). *Eclipse Mosquitto: An open source MQTT broker*. https://mosquitto.org/

Grafana Labs. (s.f.). *Grafana documentation*. https://grafana.com/docs/grafana/latest/

HiveMQ. (s.f.). *MQTT Essentials*. https://www.hivemq.com/mqtt/

InfluxData. (s.f.). *Telegraf documentation*. https://docs.influxdata.com/telegraf/

Kleppmann, M. (2017). *Designing data-intensive applications*. O'Reilly Media.

Microsoft. (s.f.). *Documentación de Power BI* [Microsoft Learn]. https://learn.microsoft.com/power-bi/

Microsoft Learn. (s.f.). *Row-level security (RLS) with Power BI*. https://learn.microsoft.com/power-bi/

OASIS. (2019). *MQTT Version 5.0* (OASIS Standard). https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

Timescale. (s.f.). *TimescaleDB documentation*. https://docs.timescale.com/